OpenAI

Subject of this case study. The logo identifies the company only. This is an independent analysis of a public post, not an OpenAI publication.

AI PRODUCT MANAGEMENT CASE STUDY

Inside OpenAI's In-House Data Agent

How a context-first design turns natural-language questions into trusted answers across 70,000 datasets

Source: openai.com/index/inside-our-in-house-data-agent | Published January 29, 2026 | Case study dated November 9, 2026 | Prepared by Krishnamurthi

At a glance

Product	An internal-only agent that lets OpenAI employees ask data questions in natural language, explore datasets, run SQL and publish reports.
Core idea	Accuracy comes from layered context about tables, code and company knowledge, plus a loop that checks and corrects its own work.
Scale	More than 3.5k internal data users, over 600 petabytes, 70k datasets (data platform figures from the post).
Key lessons	Fewer, consolidated tools; guide the goal, not the path; a table's meaning lives in the code that builds it.
Verdict	The transferable assets are the context pipeline and the evaluation harness, not the chat window.

How to read this case study

Every section separates three kinds of statement, so a reader can tell the company's facts from the author's reasoning.

> **DISCLOSED** stated in the source post
>
> Facts stated in OpenAI's post, in the author's own words. The post is the only primary source.

> **INTERPRETATION** the author's analysis
>
> The author's analysis, inference or proposal. Treat it as a hypothesis to test.

> **UNKNOWN** not disclosed, so not guessed
>
> Things the post does not say, such as accuracy, retention, conversion, cost, revenue or CAC. No numbers are invented for these.

The story in one page

OpenAI did not make its data agent better by making the model smarter. It made the agent better informed.

Picture a new analyst at a company with 70,000 datasets. Ask them for last month's weekly active users and they face a quiet trap: a dozen tables look right, and each counts a slightly different population. Pick the wrong one and the number is wrong, with no error message to warn them. This is the daily reality the post describes, and it is the problem OpenAI chose to solve.

The team's answer was not a cleverer query writer. It was a **context pipeline**. Before the agent writes a line of SQL, it draws on six layers of knowledge: how tables are used, what experts have annotated, what the code that builds each table reveals, what Slack and internal documents say, what past corrections taught it, and what the live warehouse shows right now. Then it works like a colleague, not a vending machine: it plans, runs a query, notices when a result looks wrong, investigates and tries again, and shows its assumptions alongside every answer.

Around that core sit two quieter decisions that make it trustworthy. Access is **pass-through**, so the agent can never show someone data they could not already see. And quality is guarded by **evals**, a set of questions with known-correct SQL that act like unit tests, so improvements do not silently break what already worked.

For a product manager, the lesson travels well beyond data tools. In an agent product, the durable advantage is often the context you assemble and the trust you build around it, not the chat window on top. The rest of this case study takes the post apart in twelve sections, then asks whether the idea can be rebuilt for a small business that has no data team at all.

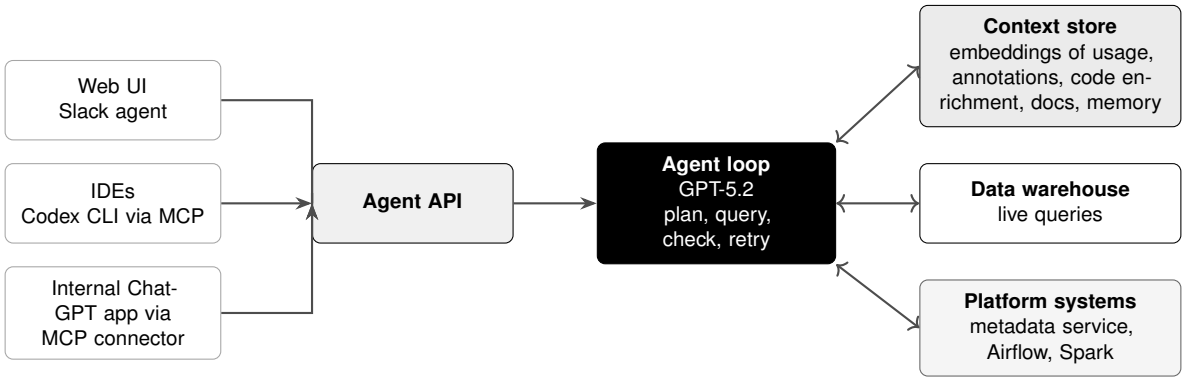


Figure 1: System architecture overview, redrawn from the post's description. Many entry points feed one agent service. The agent reasons with GPT-5.2 and draws on pre-built context, the live warehouse and other platform systems.

1 Context

At OpenAI's scale, finding the right table and using it correctly is the bottleneck, not writing SQL.

DISCLOSED stated in the source post

- The data platform serves more than 3.5k internal users across Engineering, Product and Research, covering over 600 petabytes in 70k datasets.
- The team built a custom, internal-only agent. The post stresses it is not an external offering and is built around OpenAI's data, permissions and workflows.
- It is powered by GPT-5.2 and built with Codex, the Evals API and the Embeddings API, which the post notes are available to developers.
- The post was written by Bonnie Xu, Aravind Suresh and Emma Tang and published on January 29, 2026.

INTERPRETATION the author's analysis

Turning a business question into a trustworthy data answer is a problem every large company has, and warehouse vendors already sell natural-language analytics. The post also works as a showcase for OpenAI's own developer tools, which is a reasonable reading of why the build is described in such detail.

UNKNOWN not disclosed, so not guessed

Whether OpenAI plans to productise the agent, and how much engineering effort went into it.

2 Target user

Two kinds of people feel this problem differently: analysts who can verify SQL, and operators who cannot.

DISCLOSED stated in the source post

Teams across Engineering, Data Science, Go-To-Market, Finance and Research use the agent. The post says it lowers the bar for pulling data and doing nuanced analysis across all functions, not only for the data team. Example uses include evaluating launches and understanding business health.

INTERPRETATION the author's analysis

	Analyst or data scientist	Operator (Finance, GTM, Research)
Wants	Speed, correct joins, reusable analysis	A quick answer to a business question
Can verify SQL?	Yes	Usually not
Needs from product	Inspectable steps, control, fewer debugging hours	Trust without inspection

The post's trust features, such as links to the underlying query results, mostly serve the first group.

UNKNOWN not disclosed, so not guessed

The split of usage between the two groups, and adoption per function.

3 Problem discovery

The pain sits before the SQL is written: choosing among near-identical tables, then avoiding silent logic errors.

DISCLOSED stated in the source post

- One internal user says there are many similar tables, with differences such as including logged-out users or not, and that they spend a lot of time working out which one to use.
- Even with the right table, many-to-many joins, filter pushdown errors and unhandled nulls can silently invalidate results.
- The post shows a 180+ line query as an example of how hard it is to know the right tables and columns are used.
- The stated outcome is going from question to insight in minutes, not days.

INTERPRETATION the author's analysis

Today's workflow is: find a table, learn its quirks, write SQL, validate, share. The time sinks are discovery and semantics, not syntax. That explains why the product invests in context rather than in a smarter SQL generator.

UNKNOWN not disclosed, so not guessed

How the team found and sized the problem (interviews, support load, telemetry) and the baseline time to answer.

4 Product thesis

Accurate data answers come from rich context, so OpenAI built a context pipeline around its own data, permissions and workflows.

DISCLOSED stated in the source post

The post says high-quality answers depend on rich, accurate context, and that without it even strong models can vastly misestimate user counts or misread internal terminology. It describes the agent as bespoke, and its third lesson is that a table's meaning lives in the code that produces it.

INTERPRETATION the author's analysis

A general chatbot would lack four things, and they explain the case for building:

1. **Code-level table knowledge**, derived by crawling the codebase.
2. **Institutional knowledge** from Slack, Google Docs and Notion, with permissions preserved.
3. **Inherited access control**, so the agent never widens what a user can see.
4. **A company-specific learning loop**, where corrections become saved memories.

All four come from proprietary context, not from a different model.

UNKNOWN not disclosed, so not guessed

Alternatives considered (buying a tool, hand-building a semantic layer), build cost and team size.

5 Product experience

The experience is a conversation that runs the whole analysis and shows its work, not a one-shot query box.

DISCLOSED stated in the source post

1. The user asks an open-ended question in Slack, a web UI, an IDE, the Codex CLI through MCP, or OpenAI's internal ChatGPT app through an MCP connector. Alternatively they start a saved workflow, such as a weekly business report or a table validation.
2. If the question is unclear, the agent asks. If nobody answers, it applies defaults, for example the last 7 or 30 days.
3. The agent searches internal context, inspects schemas, runs SQL and checks its own intermediate results.
4. The answer arrives with a summary of assumptions and steps and links to the underlying results. The agent can also publish notebooks and reports.
5. The user follows up, interrupts or corrects. The agent carries full context across turns and offers to save corrections to memory.

The post's example prompt asks which NYC taxi pickup-to-dropoff ZIP pairs are the most unreliable, with the largest gap between typical and worst-case travel times, and when that variability occurs. The agent handles it end to end.
In the post's memory example, the same question about ChatGPT Image Gen logged-in DAU is shown running for about 22 minutes without memory and about 1 minute 22 seconds with it.

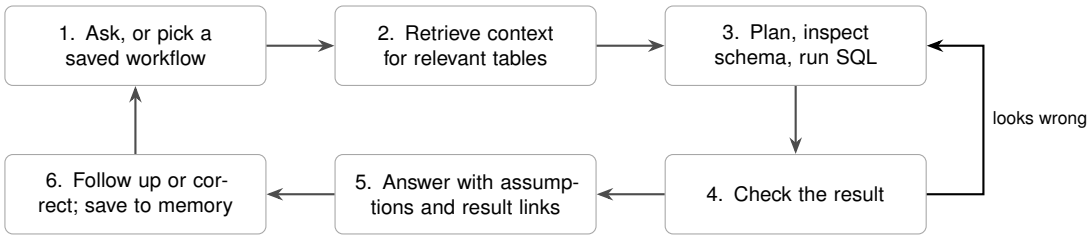


Figure 2: The agent's working loop, redrawn as an original figure from the process the post describes. The amber arrow is the self-correction path.

INTERPRETATION the author's analysis

The design goal is a teammate you can steer, which fits the exploratory questions analysts actually ask. Defaults keep the agent moving, and interruption keeps the user in control.

UNKNOWN not disclosed, so not guessed

Latency distribution, how often it asks clarifying questions, and how widely saved workflows are used.

6 AI architecture

A reasoning model drives a closed loop of tools, grounded by six context layers prepared offline and retrieved at query time.

DISCLOSED stated in the source post

Six layers of context.

Layer	What it holds	Problem it solves
1. Table usage	Schema metadata, lineage, historical queries	Which tables exist, relate and get joined
2. Human annotations	Expert descriptions of tables and columns, caveats	Business meaning schemas do not show
3. Codex enrichment	Code-derived purpose, grain, primary keys, freshness, downstream use	Telling look-alike tables apart
4. Institutional knowledge	Slack, Google Docs, Notion, ingested with permissions	Launches, incidents, codenames, metric definitions
5. Memory	Saved corrections and filters, global or personal	Repeating the same mistake
6. Runtime context	Live warehouse queries, metadata service, Airflow, Spark	Missing or stale context

- **Model and loop.** GPT-5.2 evaluates its own progress and investigates when a result looks wrong, such as zero rows from a bad join.
- **Retrieval.** A daily offline pipeline merges usage, annotations and Codex enrichment into one normalized representation. It is embedded with the Embeddings API and retrieved with RAG at query time.
- **Memory.** Corrections are saved at global or personal scope. The agent prompts the user to save them, and users can edit memories.
- **Evaluation.** Curated question and golden-SQL pairs. The generated SQL is run, and the Evals API grader compares both the SQL and the resulting data, returning a score with an explanation. The post compares this to unit tests and production canaries.
- **Security.** Access is pass-through: users can only query tables they already have permission for.
- **Build lessons.** Overlapping tools confused the agent, so tools were consolidated. Prescriptive prompts degraded results, so the team moved to goal-level guidance.

INTERPRETATION the author's analysis

Layers 1 and 2 resemble what data catalogs already offer. Layer 3, deriving a table's meaning from the code that builds it and refreshing that automatically, is the distinctive piece, and it matches the post's own lesson that meaning lives in code.

UNKNOWN not disclosed, so not guessed

Model routing, number of tools, retrieval ranking, how memories conflict or expire, and cost per answer.

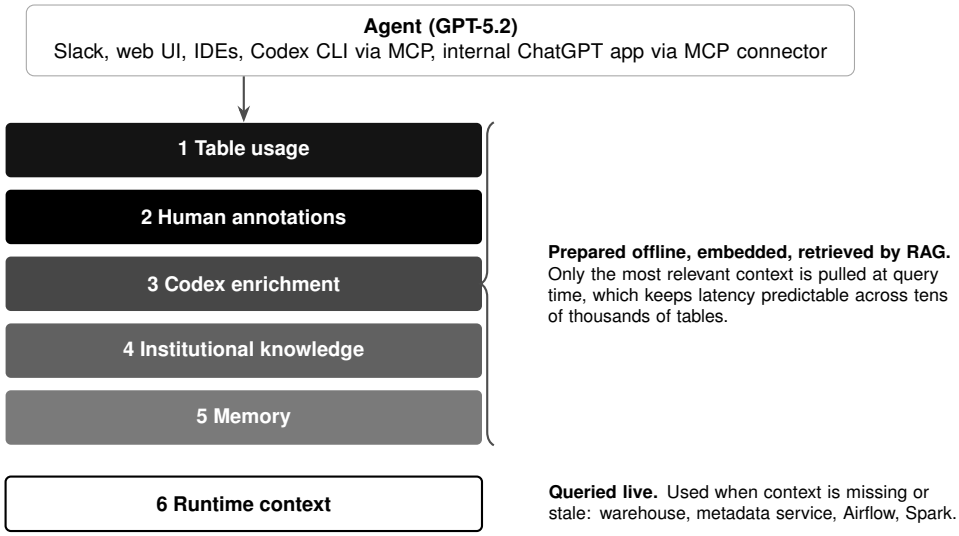


Figure 3: The six context layers and how they reach the agent, redrawn as an original figure from the post's description.

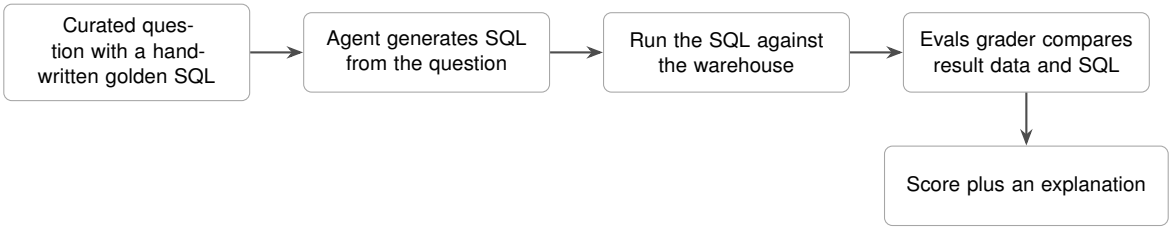


Figure 4: The evaluation loop, redrawn as an original figure. Comparing data as well as SQL allows different but correct queries to pass.

7 Trade-offs

The post makes visible choices on speed, control and flexibility, but is silent on cost.

Trade-off	Disclosed	Interpretation
Quality vs cost	Quality is protected by context layers and continuous evals. No cost figures are given.	Rich context and long agentic runs likely raise cost per answer, but this is unknown.
Speed vs reasoning	Context is precomputed daily and retrieved by RAG for predictable latency. The agent still queries the warehouse live when context is missing or stale.	Daily preprocessing may lag fast-changing tables, which live queries then cover.
Autonomy vs control	Defaults keep the agent non-blocking. Users can interrupt. Memories need user confirmation. Access is pass-through.	Autonomy is high on analysis steps and low on permissions and learning.
Flexibility vs reliability	Overlapping tools confused the agent, so they were consolidated. Rigid prompts degraded results.	Fewer tools plus goal-level guidance was the team's route to reliability.
Speed vs accuracy	Memory cut an example query from about 22 minutes to about 1 minute 22 seconds.	Context both speeds up and improves answers, so this is not a pure trade-off.

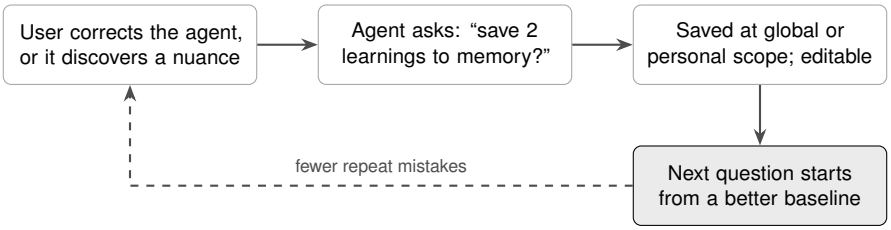


Figure 5: The memory loop, redrawn as an original figure. Corrections are stored for reuse instead of being rediscovered each time.

8 Competition

Natural-language analytics is a crowded market. Competitors mostly differ in how they obtain context and which stacks they cover.

DISCLOSED stated in the source post

The post names no competitors and makes no comparison with other tools.

INTERPRETATION the author's analysis

Alternatives, drawn from vendor-written pages. Treat their claims as vendor-reported.

Alternative	Approach	Contrast with OpenAI's agent
Snowflake Cortex Analyst	Natural language over a hand-built semantic model; Snowflake only, per Querio	Context is written by humans, not derived from code
Databricks Genie	Chat in curated Genie spaces on Unity Catalog; Databricks only, per Querio	Tied to one platform and curated spaces
Cross-warehouse BI (ThoughtSpot, Sigma, Omni, Hex, Querio)	Governed semantic layers across several warehouses	Broader reach, but semantic layers still need modeling
Power BI with Copilot, Looker	Copilot or agent over a DAX or LookML model	Strongest inside one vendor ecosystem

The thesis matches third-party evidence: Snowflake's benchmark, as reported by Atlan, lifted the same LLM from 57% to 78% accuracy by adding a semantic model. Atlan sells context tooling, so the figure is vendor-reported.

UNKNOWN not disclosed, so not guessed

How OpenAI's agent compares with any of these on accuracy, cost or setup effort.

9 Product gaps

The biggest unresolved issue is proof: the design is explained, but no measured accuracy is given.

DISCLOSED stated in the source post

The team says it is working on handling ambiguous questions better, stronger validations and deeper integration into workflows.

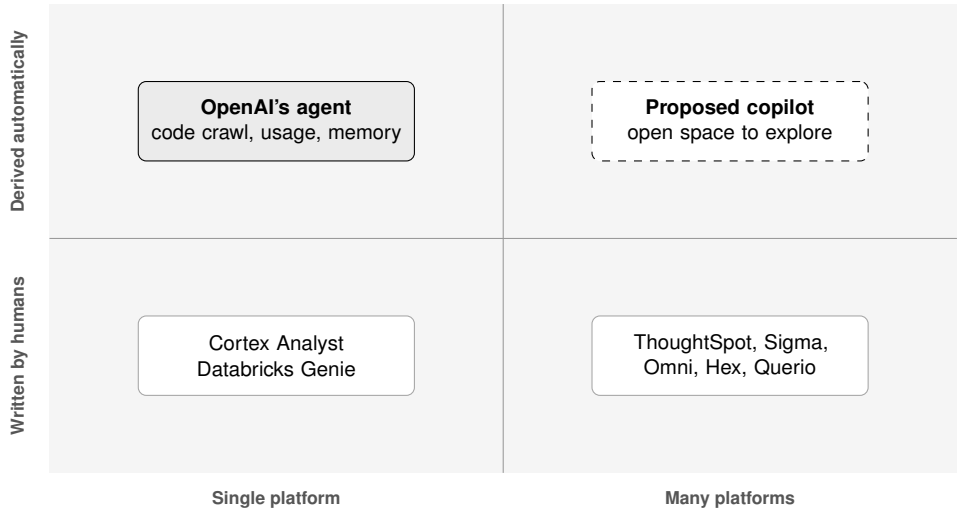


Figure 6: Illustrative positioning map. Placement is the author's judgment from the post and vendor pages, not measured data. The top-right quadrant is the opening this case study explores.

INTERPRETATION the author's analysis

- **Silent errors.** A zero-row check catches loud failures. Plausible-looking wrong joins and null handling depend on upstream context, evals and user inspection.
- **Non-analyst users.** Showing the query helps people who can read SQL. It helps less for staff who cannot.
- **Eval coverage.** Evals target important metrics and patterns, so the long tail of ad hoc questions is likely less protected.
- **Memory governance.** No detail on who approves global memories or how outdated ones are retired.

UNKNOWN not disclosed, so not guessed

Accuracy or eval pass rates; agent adoption (the 3.5k figure counts data platform users, not agent users); retention; time saved; cost per answer; latency.

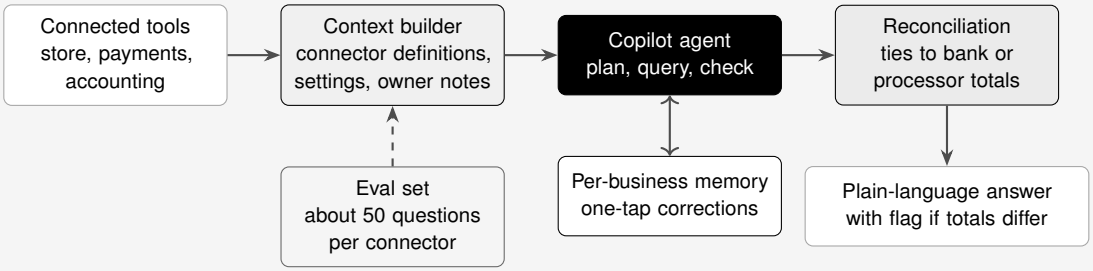
10 Your proposal: an AI BI Copilot for small businesses

Shrink OpenAI's context stack to fit a business with a few tools and no analyst, and earn trust through reconciliation, not inspection.

INTERPRETATION the author's analysis

Everything in this section is the author's proposal, not a claim from the post. The gaps in Section 9 set the design: reconciliation checks for silent errors, plain-language explanations for users who cannot read SQL, and published eval scores so accuracy is not taken on faith.

OpenAI element	Small-business version
Table usage and annotations (layers 1, 2)	Definitions shipped per connector, such as what "order", "refund" and "revenue" mean in a given store or accounting tool
Codex enrichment (layer 3)	No code crawl. Read connector settings at onboarding: currency, tax, fiscal year, sales channels
Institutional knowledge (layer 4)	A short onboarding interview and owner notes, such as "wholesale is tracked separately"
Memory (layer 5)	Per-business corrections, with a one-tap "this number is wrong"
Runtime context (layer 6)	Live read-only queries against connected tools or a synced copy
Evals	About 50 canonical questions per connector with known answers, run on every release
Show the work	Plain-language "how I got this" plus the source rows, with SQL hidden by default
Workflows	Weekly owner brief and a month-end check



Proposed architecture for the small-business copilot (author's design).

The key difference from OpenAI's case. OpenAI's users can read SQL and spot a bad join. A shop owner cannot. Trust therefore has to come from reconciliation: totals that tie to the payment processor or bank, with a visible flag when they do not.
Suggested MVP. One vertical (for example online retail), two or three connected sources, and ten killer questions such as weekly revenue, repeat-customer rate, top products by margin and refunds trend. Natural-language BI is crowded, so the wedge is accuracy plus a vertical, not the chat interface.
Top risks. Wrong money numbers destroy trust fast, connector upkeep is ongoing work, and owners must be comfortable granting data access.

11 Metrics

The north star is verified-correct answers per weekly active owner, because raw volume would reward the wrong thing.

DISCLOSED stated in the source post

OpenAI publishes no success metrics for the agent. It describes evals as unit tests and production canaries but gives no scores.

INTERPRETATION the author's analysis

Proposed for the small-business copilot. No targets are set: they should come from a pilot baseline.

Group	Metric	What it tells us
Quality	Golden-set accuracy per release	Regressions before users see them
Quality	Reconciliation pass rate	Whether answers tie to payment or bank totals
Quality	Share of answers an owner confirms correct	Real-world accuracy
Engagement	Weekly active owners, questions per owner	Habit formation
Engagement	Weekly brief open rate	Whether proactive output is wanted
Learning	Memory saves per owner	Whether corrections are being captured
Speed	Time to first useful answer	Onboarding friction
Business	Free-to-paid conversion; retention at 4 and 12 weeks	Value and stickiness
Business	Cost per answer; gross margin	Whether agentic runs are affordable

UNKNOWN not disclosed, so not guessed

Everything OpenAI measured: adoption, accuracy, retention and time saved.

12 Go-to-market

OpenAI has no go-to-market for this agent, so two cases are worth examining: a hypothetical OpenAI product, and the small-business copilot.

DISCLOSED stated in the source post

The post describes an internal-only tool, not an external offering. It discloses no buyers, pricing, distribution, revenue or CAC.

INTERPRETATION the author's analysis

Case A: if OpenAI commercialised it

The buyer would be a data or analytics leader at a large company. The product changes shape.

Area	Inside OpenAI (per the post)	What breaks for customers
Code context	Codex crawls OpenAI's own codebase	Needs access to each customer's pipeline code, a hard security ask, or a substitute
Permissions	Plugs into one existing access model	Must map each customer's warehouse and app permissions
Annotations and memory	Experts and heavy users build them over time	Cold start: value has to show in week one
Evals	Internal team writes golden SQL for key metrics	Customers must supply or co-author golden questions
Stack	One data platform	Many warehouses, tools and SQL dialects
Cost	Absorbed internally	Long agentic runs need a pricing model

The opening is automating what competing tools make humans write by hand: deriving table meaning from code and usage across more than one warehouse. Risks are customer trust in code access, per-customer setup cost, and platform vendors bundling similar features.

Case B: the small-business copilot

- **Buyer.** Owner-operators of small businesses, who are also the users. A second buyer is a bookkeeper or accountant serving many small clients.
- **Distribution.** Connector listings in the app stores of the tools it reads from, plus delivery of the weekly brief by email or a messaging app owners already use.
- **Monetisation.** A subscription per business, tiered by connectors and number of questions, with a free tier that gives the weekly brief, and a multi-client plan for accountants.

UNKNOWN not disclosed, so not guessed

Willingness to pay, price points, CAC and channel conversion. A pilot with a handful of owners in one vertical would be the first test.

Five lessons for AI product managers

INTERPRETATION the author's analysis

These are the author's takeaways from the case, not statements from the post.

1. **Context is the moat.** The same model gives different answers depending on what it knows about your data. Invest in the context pipeline before the interface.
2. **Show the work.** A visible chain of assumptions and links to results turns a plausible answer into a checkable one.
3. **Inherit, do not reinvent, permissions.** Pass-through access removed a whole class of risk and rollout friction.
4. **Evals are product infrastructure.** Golden questions let a team improve an always-on agent without fearing silent regressions.
5. **Match trust to the user.** Inspection works for analysts. For everyone else, trust has to come from reconciliation and clear flags.

Discussion questions

1. The agent's accuracy depends on context, not the model. Which of the six layers would you build first for a company with no data catalog, and why?
2. OpenAI shows its work so users can verify answers. How should that change for users who cannot read SQL?
3. The post publishes no accuracy figure. What would you need to see before trusting this agent on a board-level number?
4. Memory made one query about 17 times faster in the post's example. What governance would you add before letting users write to a shared memory?
5. If you had to charge for this product, what would you price on, and what cost driver worries you most?
6. Where would this design break first if moved from OpenAI's data stack to a customer with a very different one?

Sources and method

Primary source (read in full). OpenAI, "Inside OpenAI's in-house data agent," January 29, 2026. https://openai.com/index/inside-our-in-house-data-agent/

Secondary sources (read in full, vendor-written).

- Atlan, "Cortex Analyst vs Custom Text-to-SQL," July 29, 2026. https://atlan.com/know/snowflake/cortex-analyst-vs-text-to-sql/
- Querio, "Best AI Analytics Tools That Connect to Snowflake, BigQuery, and Databricks," July 25, 2026. https://querio.ai/articles/best-ai-analytics-tools-connect-snowflake-bigquery-databricks

Method note. Statements labelled Disclosed come only from the OpenAI post. The figures in this document are original redrawings of the process and architecture the post describes, plus the author's own proposals, and are labelled as such. The post's own screenshots and diagrams are not reproduced; readers who want them should view the original post. The OpenAI logo on the cover was supplied by the author and is used only to identify the subject company. This document is an independent analysis and is not an OpenAI publication. The 22-minute and 1-minute-22-second times are as shown in the post's screenshots. Vendor-sourced competitor claims are labelled as such and were not independently verified.